What a neural flow surrogate buys is near-wall representation, and the scoring measure decides the ranking

Ali Jabbary
Department of Mechanical Engineering, Urmia University
st_a.jabbary@urmia.ac.ir

Kasra Ghanavati
School of Computing and Mathematical Sciences, University of Greenwich
kg1111r@gre.ac.uk

Abstract

Machine-learning surrogates for simulation are almost universally scored by resampling their predictions onto a uniform grid. We measure what that resampling costs. On a standard external-aerodynamics benchmark, kernel interpolation over seven scalars parsed from the case name—no network, no flow-field learning—gives 8.4× lower volume-pressure error than a matched-budget Transolver on an area-uniform 128^2 raster. Re-weighting the identical predictions by the dataset's own node measure takes that ratio to 0.44×, and scoring per node on the native cloud with no rasterisation anywhere takes it to 0.21×: one pair of predictions, three defensible measures, and the ranking reverses. At native resolution the interpolator loses on **every** channel—144× on streamwise velocity, 134× on cross-stream velocity, 4.9× on pressure—and an oracle shown the test answer, allowed the single best of all 800 training fields, is 344× worse inside the first 0.005 chord, where a three-case probe of the exact least-squares bound over the whole family still gives 21×. The gap at the wall is representational, not a tuning deficit. The raster cannot adjudicate it for anyone: its own round-trip error at those nodes exceeds the surrogate's by 413× pooled and 495× near the wall. What both measures do resolve is where each method's error lives: three orders of magnitude of separation from the wall outward, agreeing in size and differing only in the far-field sign on one channel. A neural surrogate earns the boundary layer, and the protocol that reports it cannot see there.

Keywords: surrogate models, discretisation error, evaluation metrics, neural operators, solution verification, benchmarking, computational fluid dynamics.

1 Introduction

Classical CFD resolves the governing PDEs by iterating a discretised system until its residuals fall below a tolerance. It is accurate and trusted, but slow: meshing and a converged steady-RANS solve over a new geometry can take minutes to hours. Neural surrogates promise the opposite trade—millisecond inference—and a vigorous literature now maps geometry plus boundary conditions directly to flow fields with neural operators and attention models.

That promise is assessed almost entirely against other learned models, and through a scoring path nobody has measured. AirfRANS (Bonnet et al., 2022), the standard 2-D external-aerodynamics benchmark, distributes wall-clustered unstructured point clouds and has accumulated a leaderboard of graph networks, point-cloud operators and physics-attention transformers, each reported against its predecessors. Field errors are conventionally computed by resampling predictions onto a uniform raster and averaging over its cells. Two things are missing from that arrangement: a *non-learned* reference point—which the dataset hands over for free, because every case is indexed by a design vector printed in its own file name and parsed by its own loader (inlet velocity, angle of attack, four NACA digits)—and any statement of what the resampling costs.

Research question. *What does learning add, on this benchmark, over interpolating between cases in that parameter space—and can the protocol that reports field errors see whatever it adds?*

What a surrogate buys is near-wall representation, and no interpolation reaches it. We answer the first half at the resolution the data actually has. Gaussian kernel-ridge interpolation over seven scalars derived from the case name—no network, no geometry encoder, no flow-field learning—is evaluated against a matched-budget Transolver at AirfRANS's own cloud nodes, all 200 `full` test cases, 35.8 M nodes, with **no rasterisation anywhere in the comparison path**. Each training field is carried on its own native cloud, so neither arm meets a grid. **The interpolator loses on every channel**: **144**× on streamwise velocity, **134**× on cross-stream velocity, **4.9**× on pressure and 18× on the eddy viscosity, and 24.7× over u, v, p in standardised units; the surrogate is better on 200/200 cases on three channels and 142/200 on pressure (Table 1). The pre-registered rule returns NOT-COMPETITIVE-AT-NATIVE. Nor is this a tuning deficit that a better weighting would close. An *oracle* that is shown the test answer and allowed to pick the single best of all 800 training fields is still 344× worse than the surrogate inside the first 0.005 chord; and because a minimum over single fields bounds no combination of them, we compute the unconstrained least-squares projection of the true field onto the span of all 800—the exact lower bound for *any* weighting of that family, convex or not—which is still **21**× worse there (FAMILY-BOUND-CLOSED; three cases, u, the band holding 41% of every cloud's nodes). The gap at the wall is representational. The mechanism is measured rather than argued: inside 0.005c a query sits on average 7.8 times further from the training case's wall than from its own, and 35% of the weight mass lands *inside* the training airfoil.

The published field-MSE ranking is decided by a weighting nobody states. That verdict inverts the one the benchmark's own protocol returns, and the inversion is the second finding. Scored the way this benchmark's field tables are reported—an area-uniform 128^2 raster—the same interpolator gives 8.4× *lower* volume pressure error than the same Transolver (`mse_p` 75.0 against 628.5). Re-weight the *identical* predictions on the *identical* cells by the number of native cloud nodes each cell contains, changing nothing else, and 8.4× becomes **0.44**×; per node on the native cloud it is 0.21×. One pair of predictions, one set of cases, three defensible measures, and the ranking reverses between the first and the third. Every other channel moves with it: `mse_u` from 6.2× against the interpolator to 86× against it area-uniform-to-node-weighted and 144× at the nodes, `mse_v` from 3.0× in its favour to 15× against it. Over u, v, p in standardised units the interpolator is 1.99× better area-uniform and 8.30× worse node-weighted. Neither grid measure is obviously the right one, and that is the point: the raster is what the field tables report, while AirfRANS is a boundary-layer mesh that places 64.4% of every case's nodes inside 0.05c of the wall, a region holding 1.25% of the raster's area—a factor of 51.5, rising to 312 inside 0.005c and inverting to 0.14 beyond half a chord. **No published AirfRANS field comparison states which weighting it uses** (Table 4).

The scoring raster cannot adjudicate near-wall skill for anyone. Re-weighting fixes the measure, not the representation: both grid arms are still scored on $r128$ fields, and a method that is right at sub-grid scales earns no credit under either weighting. Resampling the *rasterised ground truth* back at the native cloud nodes measures what the representation costs before any model is involved. That round-trip error exceeds Transolver's own per-node error by **413**× pooled over the scored crop and by **495**× inside 0.005c—the band holding 43% of the dataset's nodes. An $r128$ field metric on AirfRANS is a measurement of far-field skill, for the surrogate as much as for the interpolator, and we read every grid table in this paper that way. The same fact rescues the baseline from the charge that we crippled it: the published $r128$ interpolation output, resampled at those nodes, scores 400 on u against the native construction's 98, so the grid protocol was costing the *interpolator* 4.1× on u and 69.9× on p. We gave it the better representation and it still lost.

Where each method's error lives is monotonically separated. What the two arms do separate, sharply and by three orders of magnitude, is the spatial distribution of their error—and this is the one finding both measures agree on. On the grid, band by band outward from the wall, the ratio of interpolation error to Transolver error on u runs **906**× / 74× / 30× / 5.3× / 0.57× / 0.56× / **0.31**×: monotone across all seven bands, a span of 2905×, no inversions, and 71.9% of Transolver's own squared u error beyond $0.5c$ against the interpolator's 3.6%. At the native nodes the same ladder runs $173\times \to 3.8\times$ on u, $167\times \to 0.057\times$ on v and $5.4\times \to 0.0038\times$ on p, with a 4060× surface-node row the grid cannot represent at all. The two ladders agree on the separation and its size and *disagree on the sign of the far-field u entry*; we report both pre-registered verdicts, CONFIRMED on the grid and PARTIAL at the nodes, and say in subsection 4.1 why we cannot yet decompose the disagreement. The interpolator's own side of the boundary is sharp under either: 92% **of its u error and** 90% **of its v error lie within** 0.02 **chord of the wall**—0.5% of cells, thinner than one grid spacing—while 98.7% **of the domain**, everything beyond 0.05 chord, is reproduced at $R^2 \geq 0.9996$ with no learning at all. A frozen ladder to 256^2 and 512^2 leaves the domain fraction, the R^2 floor and the outer-region error unmoved but eases the near-wall shares to 88%/84%, because the near-wall error itself falls 45%; the pre-registered rule reads that easing as PARTIAL and we report it as such.

What a surrogate-side physics residual can and cannot do. The obvious response to a broken field metric is to fix it with physics: score a surrogate by how well its field satisfies the governing equations. A deployed surrogate cannot afford the solver that produced its labels, so any such check must monitor a cheaper, different operator—and we measure what that substitution costs. On 200/200 real AirfRANS cases the monitored residual of the *exact* field is substantially nonzero while a physically wrong uniform freestream scores an exact zero, and descending that residual from the exact ground truth drives the field error up on 200/200 cases (subsection 4.3). The boundary is the operator, not the problem class. We report this because it bounds a repair a reader will reach for, not because it is the headline; the full theorem and control set are deferred to a companion paper.

The gap this fills. The claim is narrow and we state it as such. Across AirfRANS (Bonnet et al., 2022), its Transolver lineage (Wu et al., 2024; Luo et al., 2025), the subsequent architecture surveys (Scherz et al., 2026; Rigotti et al., 2026) and the aerodynamic benchmarks that followed (Ashton et al., 2024; Elrefaie et al., 2024), we find *no published parameter-space interpolation baseline for AirfRANS field metrics, and no statement in any of them of the cell weighting their volume averages use.* The nearest occupant of the first gap is DrivAerNet++, which does report an AutoML tabular baseline on its 26 design parameters (Elrefaie et al., 2024)—so the broader claim that nobody publishes parameter-space baselines is false, and we do not make it. What survives is that theirs is an R^2-on-magnitude comparison on integrated coefficients rather than a field one, and that the same group's follow-on architecture benchmark drops it. The benchmark's own authors, meanwhile, already located the failure mode without quantifying it: they conclude (their page 8) that "the models have difficulties to predict the wall shear stresses as the velocity values at the closest nodes from the geometry are often largely overestimated", which "particularly affects the accuracy of the drag coefficient" (Bonnet et al., 2022). This paper supplies the measurement—the same observation from the opposite direction, that near the wall is the *only* place learning helps, and that the protocol reporting it cannot see there.

1.1 Contributions

The findings above are the contributions, and the paragraphs that state them carry their numbers; we do not restate them here. What each adds beyond the headline number is:

1. **A head-to-head at native resolution, and a closed bound on the baseline's whole family (headline, subsection 4.1).** The comparison is run where the data lives, with no raster in the path and both arms gated against their published rows—the interpolation arm reproduces its weight matrix to 1.6×10^{-8} relative, the surrogate arm reproduces the deployed backbone at relative error exactly 0.00e+00. The rule was committed before the first number existed and returned NOT-COMPETITIVE-AT-NATIVE on all three of u, v, p at once. The oracle and least-squares controls then close the standing escape—"re-select its hyperparameters on the node measure"—by bounding what *any* weighting of the 800 training fields can achieve at the wall, with no free parameters at all.

2. **The aggregate field ranking is a property of the measure, not of the methods (subsection 4.1).** It is not a stable quantity in either direction: across five discretisations of the node

measure `mse_p` ranges over $[0.43, 1.61]$ against 8.39 area-uniform and 0.21 per node, with no construction leaving the interpolator an aggregate win. The re-weighting is conservative—grid binning cannot place node mass inside a sub-cell band, and solid-binned wall nodes are dropped—so the true node measure is more adverse to it than the number we quote.

3. **A representation ceiling that bounds every published $r128$ field number on this benchmark, ours included (subsection 4.1).** The scoring raster's own round-trip error at the native nodes exceeds the surrogate's per-node error by 413× pooled and 495× near the wall. This is symmetric: not a concession about one arm but a limit on what an $r128$ AirfRANS field metric can say about any method, and it is why the native-resolution head-to-head had to be run rather than refined in place.

4. **What a surrogate-side physics residual can and cannot do (subsection 4.3).** The universal form of the negative is about **iterate selection**—the residual-selected iterate is strictly worse than the error-optimal one in 24/24 cases—and we explicitly do *not* claim descent always harms. The boundary is the operator, not the problem class: where the monitored residual is the *solver's own*, correction works (Lei et al., 2026).

Every decision rule above was registered in a script docstring and committed before the run it governs, with each recorded amendment named and scoped where it applies. We are explicit throughout about what is **measured** versus **assumed**. The software that produces all of it is released as a CPU-first open-source package (`neuroforge-cfd`) with a frozen I/O contract, an AirfRANS loader and a per-claim reproduction map, so every number here runs from a committed script.

2 Related Work

Neural operators and physics-attention surrogates. Fourier Neural Operators learn a resolution-agnostic mapping between function spaces by parameterising a global convolution in the spectral domain (Li et al., 2021); Geo-FNO (Li et al., 2023) extends this to irregular geometries. DoMINO (Ranade et al., 2025) is a decomposable multi-scale point-cloud operator for external aerodynamics, and Transolver (Wu et al., 2024) replaces quadratic attention with attention over learnable *physics slices*, giving linear cost and strong accuracy; Transolver++ (Luo et al., 2025) scales it to million-scale meshes. We use Transolver as the learned arm throughout, because it predicts at arbitrary query points and can therefore be scored on the native cloud without a raster—the property the head-to-head needs. "SOTA" here means its standing as a published state-of-the-art architecture at the time of our experiments (ICML 2024); an independent 2026 benchmark still places Transolver(++) among the strongest aerodynamic surrogates (Scherz et al., 2026), while newer architectures have since advanced the AirfRANS accuracy frontier (Rigotti et al., 2026). Our object is the measurement, not the architecture, and nothing here is a competitiveness claim for or against any of them.

Learned solver-correctors and the "fixer" premise. Several lines of work use the discretised residual or a coarse-solution error as a *correction objective*: learned PDE solvers with convergence guarantees (Hsieh et al., 2019), deep-equilibrium operators for steady PDEs (Marwah et al., 2023), iterative refiners (Lippe et al., 2023), learned residual-error correctors (Jha, 2024), training-free correction on the PDE residual at each rollout step (Huang & Perdikaris, 2025), and PINNs (Raissi et al., 2019), which embed the residual in the training loss. Our contribution to this line is to test the premise head-on and find it does not hold in the deployment-realistic regime: descending $J = \frac{1}{2}\|R_h\|^2$ from the exact ground truth drives the field error up on 200/200 cases (subsection 4.3). The prior successes and this negative are consistent, and what separates them is the *operator*, not the problem class. Residual-driven correction works where the monitored residual is the solver's own, hence (near-)exact at the true solution—including on steady CFD, where a Newton–Krylov correction lowers the residual by seven orders of magnitude *while* reducing field and aerodynamic error (Lei et al., 2026)—so the boundary is demonstrably not steady versus transient. It fails where the monitor is instead a surrogate-side operator that a deployed system can afford, inconsistent with the one that produced the labels.

Residual-based, data-free error signals. A related line reads the PDE residual as a reference-free error indicator, which is the repair subsection 4.3 tests. The classical answer is that residual magnitude becomes an error estimate only when weighted by an adjoint solution (Becker & Rannacher, 2001), and its neural analogue bounds a PINN's true error by its residual scaled by a stability constant (Hillebrecht & Unger, 2025); Mukherjee et al. (2026) prove the corresponding convergence statement under additional compactness assumptions. All three are conditional on the residual vanishing at the exact solution—the condition an affordable surrogate-side monitor does not meet, which is what we measure.

Benchmarks, and what they compare against. AirfRANS (Bonnet et al., 2022) provides ~ 1000 incompressible steady-RANS simulations over NACA airfoils, distributed as unstructured point clouds with `full`, `scarce`, `reynolds` and `aoa` splits. Its reported field entries—and those of the Transolver lineage (Wu et al., 2024; Luo et al., 2025) and subsequent surveys (Scherz et al., 2026; Rigotti et al., 2026)—are compared against one another and against earlier learned models, never against a model of the design parameters that index the cases; and none of them states the cell weighting under which its volume averages are taken. The same holds of the aerodynamic benchmarks that followed (Ashton et al., 2024), with the one partial exception already stated: DrivAerNet++'s AutoML tabular baseline on its 26 design parameters (Elrefaie et al., 2024).

Evaluation and reporting standards in ML for PDEs. A parallel line asks whether the reported gains in this field mean what they appear to mean. McGreivy & Hakim (2024) find that 60 of 76 surveyed ML-for-PDE papers compare against a weak baseline, and identify outcome-reporting bias as systematic rather than occasional. Our result is an instance of exactly that failure mode, made concrete on one benchmark and with a constructive replacement: a baseline that runs in fifteen minutes of CPU time, and a protocol under which the comparison is decidable at all. We do not claim published AirfRANS results are wrong. We claim they have been read against a missing reference point, through a representation that cannot resolve the region where the difference between the methods lives.

Simple-baseline audits: the lineage our reference point belongs to. The control is not a new kind of instrument. In natural-language inference, *partial-input* baselines that discard the premise recover much of the label and revised what those benchmarks were taken to measure (Poliak et al., 2018; Gururangan et al., 2018). In graph classification, Errica et al. (2020) showed by comparison against *structure-agnostic* baselines that on several benchmarks structural information was not yet being exploited—substitute *geometry* for *structure* and that is the sentence we are testing here. In recommendation, Ferrari Dacrema et al. (2019) found most reproducible neural methods matched by nearest-neighbour or linear models. The genre has since reached the natural sciences: Ahlmann-Eltze et al. (2025) report that no tested deep or foundation model for gene-perturbation effects *yet* outperforms deliberately simple linear baselines. Feng et al. (2019) supply the standing caveat on the instrument and we honour its direction: a cheap baseline that succeeds shows a benchmark is cheatable, while a cheap baseline that fails licenses nothing about how hard the benchmark is. That caveat is load-bearing here, because at native resolution our cheap baseline *fails*—so the weight of this paper rests not on the baseline beating anything but on what its failure localises, and on the three ways the grid protocol hid that localisation. One structural difference from the partial-input line is worth stating precisely. A partial-input baseline consumes a subset of what the model consumes; the design vector we read off the case name is, for the most part, not an input to any AirfRANS surrogate—it is the low-dimensional code that *generated* the geometry, published in the file name. AirfRANS is a hybrid case and we flag it as such: U and Re are literally input channels, while the NACA digits enter the surrogate only through the `sdf` they generate. Framed generally, the question is how much usable information a benchmark's own published metadata carries about its headline label (Ethayarajh et al., 2022); we answer it in the benchmark's own currency—volume MSE—rather than as an information-theoretic quantity.

The estimator is decades old, and we say so first. Gaussian kernel ridge regression over a handful of design variables is Kriging in all but name, and fitting response surfaces over design variables has been standard practice in aerodynamic design since long before this benchmark existed (Queipo et al., 2005). We therefore claim no novelty for the estimator; an aerodynamicist should read subsection 4.1 as response-surface methodology used as a benchmark control, not as a new method. What is new is the measurement—what the omission of that estimator from this benchmark's reporting conceals, and where the surviving gap lives. That is also the cleanest statement of how this work extends the closest prior art in our own subfield: McGreivy & Hakim (2024) showed that the field compares against weak *numerical* solvers; we show that it does not compare against a trivial *statistical* baseline at all, and we measure how much that conceals. The protocol we recommend is likewise not an invention but an import. Data-driven weather forecasting already mandates trivial controls of exactly this kind, reporting persistence and climatology alongside every model in its headline scorecard (Rasp et al., 2024); external-aerodynamics benchmarking does not.

2.1 Positioning

This work sits in the evaluation line rather than the architecture line, and its object is a discretisation. Its primary contribution is a measurement of what the standard scoring path costs: a non-learned reference point for AirfRANS field metrics, run once through the published raster and once at the dataset's own resolution, and the three quantities that separate the two readings—the cell weighting, the representation ceiling, and the wall-distance distribution of each method's error. The positive result is what survives all three. A learned surrogate earns the boundary layer, by two orders of magnitude, and no member of the interpolation family reaches it even when handed the answer. The secondary contribution is an audit of the repair a reader will reach for—score the field by its PDE residual instead—whose organising question is not which features a system has but whether the residual it drives is the *same discrete operator* that produced its reference data. Where it is, residual-driven correction works and is already demonstrated (Lei et al., 2026). Where an affordable surrogate-side proxy stands in for it, the truth itself fails the check. We claim no novelty for either estimator we use; what is new is the measurement, and the protocol it forces.

3 Method

This section defines only the two objects later sections measure: the fixed channel contract every arm is encoded into, and the discrete residual operator subsection 4.3 audits. Implementation details, the module map and the synthetic zero-download data generator are in the released package and its `docs/REPRODUCE.md`; no quantitative claim in this paper uses synthetic data.

3.1 Geometry encoding and the monitored operator

A `FlowCase` is encoded into a fixed channel-first stack $x \in \mathbb{R}^{7\times H\times W}$ in the frozen `INPUT_CHANNELS` order ($\texttt{sdf}, \texttt{mask}, x, y, u_{\text{in}}, v_{\text{in}}, \log \text{Re}$); the backbone outputs `OUTPUT_CHANNELS` (u, v, p, ν_t), where p is the **kinematic** pressure p/ρ. Fixing this contract is what makes the backbones interchangeable and the residual monitor below backbone-agnostic.

The verifier evaluates the steady, incompressible, 2-D RANS equations in primitive form on the **physical (denormalised)** fields, with effective viscosity $\nu_{\text{eff}} = \nu + \nu_t$, pointwise:

$$\text{continuity:} \quad r_c = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}, \tag{1}$$

$$x\text{-momentum:} \quad r_x = u\,\frac{\partial u}{\partial x} + v\,\frac{\partial u}{\partial y} + \frac{\partial p}{\partial x} - \nu_{\text{eff}}\,\nabla^2 u, \tag{2}$$

$$y\text{-momentum:} \quad r_y = u\,\frac{\partial v}{\partial x} + v\,\frac{\partial v}{\partial y} + \frac{\partial p}{\partial y} - \nu_{\text{eff}}\,\nabla^2 v. \tag{3}$$

Because p is kinematic, no explicit density appears. **The turbulence term is live**: over the fluid cells of the 200-case AirfRANS test split the eddy viscosity averages $\nu_t = 8.2 \times 10^{-5}$ against a laminar $\nu = 1.56 \times 10^{-5}$, so $\nu_t \approx 5.2\,\nu$ supplies $\approx 84\%$ of ν_{eff} and the monitored operator is a genuine RANS residual. The channel is learned, backbone-dependently: against ν_t's own fluid-masked variance, $R^2 = 0.996$ on the grid backbone and 0.67–0.70 on MeshGraphNet (`scripts/check_nut_learned.py`). What the operator omits is the spatially varying term $\nabla\nu_t\nabla u$. Derivatives use central finite differences with one-sided stencils at the borders; residuals are zeroed inside the solid. A boundary-condition violation map r_{bc} adds a **no-slip** term penalising velocity magnitude in the thin fluid band adjacent to the wall (weighted by $\exp(-|\mathrm{sdf}|/\ell)$, $\ell \approx 3$ cells) and a **far-field** term penalising deviation from (u_∞, v_∞) on the outer border ring.

Two implementation facts that bear on later claims. *(i) The monitor discards its own strongest no-slip cells*, because a central stencil on the fluid ring adjacent to the solid reaches into the $u = v = 0$ discontinuity; the masking is necessary but removes a mean 46% (range 35–51%) of the squared no-slip penalty—an unlogged spot check over 8 cases, which we report as such—precisely where the proximity weight is largest. *(ii) There are two Laplacian stencils in this pipeline.* The numpy monitor whose numbers we report uses the compact three-point form; a differentiable twin used elsewhere in the package uses a repeated first difference, which is wider and annihilates the period-2 mode the compact form sees. Where the choice could have mattered most, in the descent experiment of subsection 4.3, we check both operators explicitly and both fall together in 24/24 cases.

4 Experiments

All quantitative results are on **real AirfRANS**. Three measures appear in this paper and we name each one every time it is used, because the whole result is that they disagree. **(i) Area-uniform** $r128$: the AirfRANS community protocol (`evaluate_cases`), per-channel volume MSE over the fluid cells of the 128^2 crop with every cell weighted equally, plus surface-pressure MSE on the body. This is what published field tables report and it is the measure of every grid table here. **(ii) Node-weighted** $r128$: the identical cells and the identical errors, each cell weighted by the number of native cloud nodes that bin to it (Table 4). **(iii) Per node on the native cloud**: no raster anywhere, each of the 35 849 332 test nodes counted once (Table 1). Force columns, where they appear, are relative errors against the force integrated from the *rasterised ground-truth field* through the same grid integrator—a field-to-field consistency valid for ranking arms, not agreement with AirfRANS's official labels, and subsection 4.2 states what that integrator can and cannot see. The ν_t channel is omitted from the force-bearing tables because its raw MSE is not comparable to the others; it is reported in the two head-to-head tables, where it matters.

The learned arm is a Transolver (7.35M parameters) trained on AirfRANS `full` (800 train / 200 test, 80 epochs, per-point standardised MSE over a uniform subsample of each case's own cloud), reported as the mean over its three deployed seeds with per-seed values given wherever a verdict could turn on them. Both arms in every head-to-head are gated against their published rows before any new number is computed, and each gate is named where it is read.

4.1 What a surrogate buys, and the measure that decides the ranking

AirfRANS indexes every case by a short design vector that the dataset's own file name prints and its own loader parses: the fields of `airFoil2D_SST_U_alpha_digits` are the inlet velocity, the angle of attack and the four NACA digits, and `Simulation.reset()` reads the first two directly off the name. Before asking how well a neural operator predicts these flows, we ask what that index alone already determines.

The baseline. From the name alone we build seven analytic scalars—inlet velocity U, geometric incidence α, NACA maximum thickness $t_{\max}$, thin-airfoil effective incidence $\alpha - \alpha_{0L}$, and the camber line at $x/c \in \{0.25, 0.5, 0.75\}$—and interpolate the *nondimensional* field $(\hat{u}, \hat{v}, \hat{p}, \hat{\nu}_t) = ((u - U\cos\alpha)/U,\ (v - U\sin\alpha)/U,\ p/U^2,\ \nu_t/(Uc))$ across training cases with Gaussian kernel ridge regression, redimensionalising with the test case's own (U, α). There is no network, no geometry encoder and no flow-field learning; fitting is one 800×800 Cholesky solve and prediction is one GEMM. The estimator is deliberately old—this is response-surface methodology, and Gaussian kernel ridge over design variables is Kriging in all but name (Queipo et al., 2005)—so the claim we make is for the measurement, not for the method. Hyperparameters come from 5-fold cross-validation *inside* the train split on a scalar registered before any number existed, applied to test once. Scoring is byte-identical to the surrogates': the same cached rasterised pairs at 128^2, the same `evaluate_cases`, the same mask, surface sampler and force integrator, with `sdf` and `mask` rebuilt from `case.geometry` exactly as the Transolver adapter does (`scripts/parameter_interpolation_baseline.py`, `results/interpolation/interp_full.json`). The input asymmetry runs *against* the baseline: the surrogates receive the full 7-channel geometry and freestream encoding per case, the interpolator receives seven scalars.

The *objective* asymmetry runs the other way, and it is large enough to decide the comparison. The interpolator is selected by cross-validation on the area-uniform grid measure the published tables report; the surrogate minimises a per-point loss under the dataset's own node distribution, which places 55.5% of its nodes inside $0.02c$ against 0.51% of the raster's cells—a factor of 109.5, rising to 312× inside $0.005c$ and inverting to 0.14× beyond $0.5c$. One arm is therefore tuned on the scoreboard and the other is not, so we report the head-to-head under all three measures of section 4 and lead with the one that has no raster in it at all: Table 1 scores both arms per node on the native cloud, Table 2 gives the published area-uniform grid comparison, and Table 4 re-weights that grid comparison by node count without changing a single prediction.

The head-to-head with no raster in it, and the interpolator loses every channel. The published interpolant is a weight matrix over the 800 training cases applied to the nondimensional field and redimensionalised with the test case's own (U, α); the weights depend only on the seven name-parsed scalars, so nothing about them is grid-specific. What is grid-specific in the published run is the *representation* of the fields being combined. We therefore evaluate the identical estimator with each training field carried on **its own native cloud**, by the same Delaunay-barycentric interpolant the loader uses to build every rasterised pair in this project, and query it at the test case's own nodes; the Transolver is evaluated at those same nodes directly, which is where it natively predicts. No new numerical operator enters the comparison and neither arm meets a grid (`scripts/point_space_headtohead.py`; `results/interpolation/point_space_headtohead.json`). Two gates make this the published pair rather than a re-implementation: the rebuilt weight matrix reproduces `interp_full.json` to $1.8 \times 10^{-10}/2.4 \times 10^{-9}/1.6 \times 10^{-8}$ relative on $u/v/p$ (**G1**), and the point-space Transolver reproduces the deployed backbone's own per-band per-node MSE at relative error exactly 0.00e+00 on all three seeds, all four channels and all seven bands (**G2**).

The interpolator loses on all four channels (Table 1): 144× on u, 134× on v, 4.85× on p and 18.3× on ν_t, and 24.7× over u, v, p in standardised units. The pre-registered rule—$R_c > 2$ on all three of u, v, p—returns NOT-COMPETITIVE-AT-NATIVE, and it fires with room to spare on two of them. This is not a seed artifact (R_u 147.3/141.0/144.4, R_p 5.23/4.17/5.34) and not a few-case artifact: paired per case, the surrogate is better on 200/200 cases on u, v and ν_t and on 142/200 on p (one-sided binomial $p = 1.3 \times 10^{-9}$; per-case R_p median 2.21, p10 0.44, p90 9.2). Nor does it depend on the one genuinely arbitrary choice in the construction. In-body queries—the test nodes that fall inside a *training* airfoil—admit two conventions, and the rule reads the better of the two per channel, which on p is a 28% gift to the interpolator (41 740 against 58 020, moving R_p from 6.74 to 4.85). Dropping the 201 444 surface nodes where that convention is most arbitrary leaves R_u 138.5, R_v 126.1, R_p **5.08**—the headline channel gets *worse* for the interpolator without them.

The obvious escape is closed, and closed for the whole family. The standing reply to a losing baseline is "re-select its hyperparameters on the measure you are scoring it under". We bound what that can buy, with no free parameters at all. First, an *oracle*: for each test case and band, the single best of the 800 training fields, chosen knowing the test answer. Inside 0–$0.005c$ it scores 397.7 on u against Transolver's 1.158—a factor of **343.6**, against a pre-registered threshold of 10 (NO-PARAMETER-COMBINATION). We state the weakness of that instrument before a reviewer does: a minimum over *single* fields is not a lower bound on linear *combinations* of them, and in that innermost band the fitted combination indeed beats the oracle (200.3 against 397.7), because averaging misaligned profiles cancels variance. So we compute the bound that does hold. Because the published weights sum to one, redimensionalisation commutes with the combination, and the estimator's prediction lies in the *span* of the 800 redimensionalised single-field predictions at the test nodes. The unconstrained least-squares projection of the true field onto that span, computed per band per case with the test answer in hand, is therefore a genuine lower bound on what *any* weighting of those fields—any kernel, any bandwidth, any ridge, any selection measure, convex or not—can achieve at those nodes. Inside 0–$0.005c$ on u it is still **21.3**× Transolver's band error, returning the pre-registered FAMILY-BOUND-CLOSED. **Scope, stated inline because it is narrower than the rest of the table**: this arm is $n = 3$ test cases, one channel, and the single band whose node count (73–82k) far exceeds the 800 free parameters of the projection, which is the only band where the quantity is a bound rather than

Table 1: **The head-to-head at native resolution**: per-node MSE against the native AirfRANS targets, all 200 `full` test cases, 35 849 332 nodes, *no rasterisation anywhere in the comparison path.* Both arms are the published rows—the interpolation weight matrix reproduces `interp_full.json` to $\leq 1.6 \times 10^{-8}$ relative (**G1**) and the Transolver reproduces the deployed backbone's per-node band MSE at relative error 0.00e+00 (**G2**); Transolver is the mean over its three deployed seeds, with per-seed ratios 141–147 / 129–142 / 4.2–5.3 on $u/v/p$. The interpolator is given the better of two in-body conventions on each channel. Paired per case, Transolver wins 200/200 on u, v and ν_t and 142/200 on p. **The two memo rows show that the grid protocol was costing the *interpolator*** 4.1× on u and 69.9× on p: its published $r128$ output, resampled at these same nodes, is barely distinguishable on p from the raster's own model-free round-trip error, so the native construction favours the baseline rather than handicapping it. (Memo rows are in-crop pooled, the only region the bilinear sampler reports without clamp contamination.) Source: `results/interpolation/point_space_headtohead.json`, rules **P1**–**P4** committed in `a54cb75` before any number existed.

per-node MSE, native cloud	`mse_u`	`mse_v`	`mse_p`	`mse_nut`
parameter interpolation @ native nodes	93.11	66.52	41 740	6.92×10^{-7}
Transolver @ native nodes, 3 seeds	**0.6458**	**0.4968**	**8605**	$\mathbf{3.79\times10^{-8}}$
ratio (interp / Transolver)	**144.2**	**133.9**	**4.85**	**18.3**
memo: published $r128$ interpolation, resampled at these nodes	400.2	189.7	3.06×10^{6}	2.08×10^{-6}
memo: the $r128$ raster's own round-trip error, no model	281.5	272.6	3.01×10^{6}	1.54×10^{-6}

an interpolation (`results/interpolation/point_space_oracle_ls.json`; amendment 2, registered before the stage was run and carrying no threshold of its own). The 343.6 is the 200-case number; the 21.3 is the three-case one, and neither inherits the other's sample size.

The mechanism, measured rather than argued. Parameter interpolation combines fields in *physical* coordinates, and a test node at wall distance d_t sits at some other wall distance under every training geometry. Weighting by $|W|$ over all 800 training cases: inside 0–0.005c—where 41% of every AirfRANS cloud lives—a query at a mean 0.00094c off its own wall lands on average 0.0073c from the training case's wall, **7.8**× further away than the boundary-layer position it is trying to sample, and **35**% of the weight mass lands *inside* the training airfoil altogether. The 0.0234c raster hid this by averaging over it. No extrapolation is involved—the mass falling outside any training hull is 0 in every band but the outermost, where it is 0.094%—so neither trigger of the pre-registered inconclusive rule fires and no band is INCONCLUSIVE. What we deliberately did *not* build is a body-fitted or wall-aligned variant of the interpolator: it would need a blend length scale and an arc-length correspondence with no published provenance, and a reviewer would reopen it by attacking the tuning. The least-squares bound does that job without free parameters. The honest scope of this result is therefore parameter interpolation of AirfRANS fields *in physical coordinates*, which is what the published estimator does and what the response-surface literature does; a body-fitted parameter interpolator is a different method and we do not test it.

On the published grid the verdict inverts, and the inversion is a weighting. Scored area-uniform on the 128^2 raster the field tables are reported on, the same interpolator *beats* the same Transolver on cross-flow velocity (`mse_v` 0.0336 against 0.088, 2.6×) and on volume pressure (`mse_p` 75.0 against 628.5, 8.4×). Re-weight the identical predictions on the identical cells by the number of native cloud nodes each cell contains—same truth, same fluid mask, same 200 cases, same per-case-mean protocol, *only the cell weights differ*—and both reverse: `mse_p` 6012 against 2646, a ratio of 0.44× where it was 8.39×; `mse_v` 4.26 against 0.282, 0.066× where it was 2.97× (Table 4). Before the re-weighting both arms reproduce their published rows to below one part in 10^9, the Transolver arm exactly (`scripts/measure_asymmetry.py` gates **G1**/**G2**). **One pair of predictions, one set of cases, three defensible measures, and the pressure ratio runs 8.39×, 0.44×, 0.21×.** That is the finding: the aggregate field ranking on this benchmark is not a property of the methods. The interpolator loses under *every* measure on the streamwise channel (`mse_u` 0.782 against 0.120 area-uniform, 30.7 against 0.358 node-weighted, 93.1 against 0.646 per node), marginally on surface

Table 2: **The same comparison on the published grid, where it reverses.** Parameter interpolation against the Transolver on AirfRANS `full` (800 train / 200 test, $r128$, identical `evaluate_cases`, 3 seeds where seeded), scored area-uniform. The force columns are mean relative error against the force integrated from the rasterised *ground-truth field* through the same grid integrator—a field-to-field consistency measure, not agreement with the official AirfRANS labels—and ρ_{C_l}, ρ_{C_d} are the corresponding self-consistency correlations defined in section 4. That integrator carries an $\approx 11\times$ absolute drag bias even on exact fields and cannot see viscous drag (subsection 4.2), so these four columns rank arms against one another; they do not measure force accuracy. The last block is the floor ladder and the negative control; "permuted" predicts each test case from another case's parameters under a fixed derangement, everything else identical. The interpolator's 0 parameters are not free: it carries the 800 training fields (210 MB) resident at inference. Transolver's force-rank seed standard deviations are 0.0002 (ρ_{C_l}) and 0.0012 (ρ_{C_d}), kept out of the cells for width. **Bold marks the better arm under the area-uniform cell weighting these volume columns use, and every bolded interpolation cell is un-bolded by Table 1.** `mse_v` and `mse_p` carry no bold at all because their ordering already reverses under the dataset's own node measure on these same predictions (Table 4).

model	`n_params`	`mse_u`	`mse_v`	`mse_p`	surf. `mse_p`	ρ_{C_l}	ρ_{C_d}	C_l err	C_d err
Transolver	7.35M	**0.120±0.005**	0.088±0.013	628.5±29	**9110±500**	0.9992	0.9963	5.81%	8.99%
parameter interpolation (no learning)	0 (210 MB)	0.782	0.0336	75.03	10989	0.9999	0.9991	**1.18**%	**2.39**%
floor: 8-NN, inverse distance	0	0.904	0.143	1145.5	108022	—	—	—	—
floor: 1-NN	0	1.415	0.308	2744.0	192123	—	—	—	—
floor: train-mean field	0	11.01	6.577	57980	2792467	—	—	—	—
floor: uniform freestream	0	26.01	17.65	151365	4120337	—	—	—	—
control: permuted parameters	0	22.82	14.36	129622	5939745	0.345	—	365%	367%

pressure (10989 against 9110, 1.21×), and on the closure (`mse_nut` 14.4× area-uniform, 9.2× node-weighted, 18.3× per node—the only channel whose margin narrows rather than reverses).

Because `run_baselines.py` trains on per-channel *standardised* MSE over all four output channels, we report that metric too, so the comparison cannot be dismissed as scoring a model on something it was never trained for (Table 3). Both of its means must be quoted together, and both are area-uniform: **over the three channels the tables report the interpolator is** 1.9× **better, and over all four channels of the surrogate's actual loss the surrogate is** 6.4× **better**, because ν_t dominates both models' standardised error and the interpolator cannot represent the closure at all. Recomputed from Table 4's own rows against the same $\mathrm{Var}_{\mathrm{train}}$ divisors—which are common to both arms and so cancel in every ratio—the area-uniform pair is 1.99× and 6.1×, differing from Table 3 only in the provenance of the Transolver row; the node-weighted pair reverses to **8.30**× and 8.85×, and at the native nodes the three-channel mean is **24.7**×, all in the surrogate's favour. The force columns carry no cell weighting to change, so they are untouched by any of this—but they need their name said out loud, because it is not the name a reader supplies. They measure agreement with the force integrated from the rasterised *ground-truth field* through the same grid integrator, so 4.9× and 3.8× are field-to-field consistency ratios and not better lift and drag; subsection 4.2 states what that integrator can and cannot see. One tension we record without resolving: the interpolator is 1.21× *worse* on surface-pressure MSE yet 4.9× better on the lift column, which is an integral of that same surface pressure, so the two statistics must weight the body differently and we have not decomposed which cells drive each.

"Weight by node count" is not a single number, and none of its readings rescues the aggregate. The node measure admits more than one discretisation, so we re-derive the same ratio under five of them from committed numbers, with no new inference (the `sensitivity` stage of `scripts/measure_asymmetry.py`). On `mse_p` they span $[0.43, 1.61]$ against 8.39 area-uniform, and the endpoints are not equally credible. The two faithful cell-level estimators—sample the grid error field at each node, then average over nodes, pooled or per case—give 0.431 and 0.440; the coarse but internally consistent band-level construction gives 1.10; the largest value, 1.61, comes from the two constructions that pair true node masses with cell MSEs drawn from a different spatial set, which we report because they are what a reader would compute by hand from Table 6 and because they are the least favourable to our own conclusion. **No construction leaves the interpolator an aggregate win**: the construction that reaches 1.61 on p is 551× against it on u and

Table 3: The same runs in per-channel standardised MSE—each channel divided by its own train-set variance (u 341.9, v 52.45, p 135590, ν_t 1.320×10^{-6}), which is the quantity the surrogates are trained on, with `F_OUT` = 4. The last two columns are the two means, and neither should be quoted without the other. Every cell here is **area-uniform**; the divisors are common to both arms, so the row ordering is the ordering of Table 2 and reverses with it under the node measure. Applying these same divisors to Table 4's rows gives 1.99× and 6.1× area-uniform and 8.30× and 8.85× node-weighted, both in Transolver's favour in the second case; at the native nodes of Table 1 the three-channel mean is 24.7× in Transolver's favour.

model	std. `mse_u`	std. `mse_v`	std. `mse_p`	std. `mse_nut`	mean u, v, p	mean u, v, p, ν_t
Transolver	3.51×10^{-4}	1.68×10^{-3}	4.64×10^{-3}	4.34×10^{-3}	2.22×10^{-3}	$\mathbf{2.75\times10^{-3}}$
parameter interpolation	2.29×10^{-3}	6.41×10^{-4}	5.53×10^{-4}	6.70×10^{-2}	1.16×10^{-3}	1.76×10^{-2}

Table 4: **The ranking is the weighting.** The same 200 test cases, the same rasterisation and fluid mask, the same predictions and the same per-case-mean protocol; *only the cell weights differ.* "Area-uniform" gives every fluid cell equal weight, which is what Table 2 reports. "Node-weighted" gives each fluid cell the number of native AirfRANS cloud nodes that bin to it, which is the measure the dataset itself carries. Bold marks the better arm within each row; the last two rows are Transolver over interpolation, so a value above 1 means the interpolator is better by that factor and is bolded. Before re-weighting, the interpolator arm reproduces `interp_full.json` to below one part in 10^9 and the Transolver arm reproduces `v2_results.json backbone_per_seed` exactly (gates **G1**/**G2**); Transolver is the mean over the three deployed seeds (per-seed R_p: 8.90/8.66/7.61 area-uniform, 0.418/0.433/0.469 node-weighted). The area-uniform Transolver `mse_p` is 629.6 here against 628.5 in Table 2, which transcribes `results/baselines/table2.csv`; the two differ only in provenance. The re-weighting is conservative—grid binning cannot place node mass inside a sub-cell band, and solid-binned nodes are dropped—so the true node measure is more adverse to the interpolator than the node-weighted row shows. Source: `scripts/measure_asymmetry.py`, `results/interpolation/measure_asymmetry.json`.

	`mse_u`		`mse_v`		`mse_p`		`mse_nut`	
cell weight	interp	Transolver	interp	Transolver	interp	Transolver	interp	Transolver
area-uniform (r128 crop)	0.782	**0.127**	**0.0336**	0.0999	**75.0**	629.6	8.84×10^{-8}	$\mathbf{6.15\times10^{-9}}$
node-weighted (same cells)	30.7	**0.358**	4.26	**0.282**	6012	**2646**	4.72×10^{-7}	$\mathbf{5.12\times10^{-8}}$
ratio, area-uniform	0.162		**2.97**		**8.39**		0.070	
ratio, node-weighted	0.0117		0.0663		0.440		0.108	

24× on v, and across all five node constructions `mse_u` runs 86–551× against it and `mse_v` 6.3–24×. Two properties of the re-weighting run the same way. A $0.0234c$ cell cannot sit inside a $0.005c$ band, so the grid assigns only 0.098 of the node weight to the innermost band against its true 0.432, moving mass *out of* the band where Transolver leads by 906×; and 13.4% of in-crop nodes bin to cells the geometry mask calls solid and are dropped, those being the nodes nearest the wall. The true node measure is therefore more adverse to the interpolator than 0.440, not less.

Where each arm's error lives, on the grid. Decomposing the interpolator's squared error by signed distance to the wall (`scripts/interpolation_band_control.py`, `results/interpolation/interp_band_control_full.json`) locates the entire remaining gap. **92% of its u error and 90% of its v error lie within $0.02c$ of the wall—a band holding 0.5% of cells, thinner than the $0.0234c$ grid spacing—while 98.7% of the domain lies beyond $0.05c$, where it reproduces every channel at $R^2 \geq 0.9996$** (Table 6). We report the strict per-case-centred statistic alongside the pooled one, because U spans 31–93 m/s and a pooled far-field R^2 partly rewards knowing the freestream: subtracting each case's own band mean first leaves $R^2 \geq 0.9965$ on every channel beyond $0.05c$, and moves only the sub-cell wall band ($0.840 \rightarrow 0.754$ on u). That defeats the obvious objection that the win is on easy cells—the far field is 80% of the domain and carries 3.6% of the u error and 2.2% of the p error. One scope note travels with the wall-band number: the concentration is 0.924 on u and 0.898 on

v but only 0.535 on p, so "the advantage is confined to 0.5% of cells" is a statement about the velocity channels, not about pressure.

A decomposition of one arm shows where that arm fails; it does not show that the other arm succeeds there. We therefore run the identical band accumulator on the deployed Transolver—same cases, same rasterisation, same fluid mask, reproducing Table 6 over 95 entries at a maximum relative deviation of $6.9{\times}10^{-8}$ (gate **G3**)—and report the per-band ratio of the two (Table 5). On u it runs **906**× / 74× / 30× / 5.3× / 0.57× / 0.56× / **0.31**× from the wall outward: **monotone across all seven bands, a span of 2905×, no inversions** (pre-registered rule **D2**, CONFIRMED ON BOTH ARMS), with the crossover between 0.02–0.05c and 0.05–0.15c. The two arms' error distributions are mirror images: 92.4% of the interpolator's u squared error lies inside 0.02c, while 71.9% of Transolver's lies beyond 0.5c (and 62.8% of its p error), in the 80% of cells where the interpolator carries 3.6% of its own. **That is why an aggregate field-MSE table is decided by its weighting**: the two methods are accurate in different places, by three orders of magnitude, and any aggregate is a choice about where to look. The p row makes the point concretely—Transolver better in the innermost band (0.589), the interpolator winning modestly from 0.005c to 0.05c and then by 32× and 237× in 0.15–0.5c and beyond: *the area-uniform 8.4× on `mse_p` is a far-field statement about 80% of the cells and 11% of the nodes.*

The same ladder at the native nodes, and the one place the two measures disagree. Re-running the decomposition per node on the native cloud, with no raster anywhere and with an extra row for the 201 444 surface nodes the grid cannot represent, the ratio runs 4060× (wall) / **173**× / 152× / 43.8× / 14.4× / 1.83× / 2.29× / **3.79**× on u, 2834× → 0.057× on v, and 1.50× / **5.42**× → **0.0038**× on p (all rows on the same in-body convention **P1** selected, so the ladder and the aggregate are the same arm). The span, the size and the monotone decay from the wall outward all survive the removal of the raster; on p the span is 1435× over the seven non-wall bands, and on v it is 2958×. **The surface row is the one the force claims sit on**, and it is new: at sdf = 0 the surrogate is 4060× better on u, 2834× on v and 382× on ν_t, while the two arms are within 1.51× on p (149 203 against 99 227) and both are poor there in absolute terms relative to $\mathrm{Var}_{\mathrm{train}}(p) = 135\,590$.

Two readings change and we state both. First, the far-field claim on v and p gets *larger*, not smaller: beyond 0.5c the interpolator is 17.7× better on v and 265× better on p. Second, and against us: **on u the interpolator is behind in every band at the native nodes, including the far field, where the grid measure had put it 3.2× ahead.** The pre-registered localisation rule therefore returns $S = 45.6$ with *two* inversions and reads PARTIAL here against CONFIRMED on the grid, and we report both verdicts rather than the convenient one. We can say what does *not* explain the flip: resampling the published $r128$ output at those same far-field nodes costs the interpolator only 1.27× there, far short of the ≈ 10× the swing requires, so "it was the raster" is not supported either. Two things change together and this run does not separate them—which points inside a band are sampled (area-uniform cells and native nodes are not distributed alike within a band), and what the reference is (the grid metric scores a rasterised prediction against a *rasterised* truth, so the representation error partly cancels; the node metric does not). The sub-band decomposition that would apportion them has not been run, and we make no claim about it. That the two measures agree on the separation everywhere except this one entry, and disagree there for reasons that are themselves about measure and representation, is the thesis of this section applied to its own instrument.

Two sample-size facts belong with the innermost number rather than being left for a reviewer to find. Pooled over 200 cases the 0–0.005c band holds 4489 fluid cells (22.4 per case) against 2 595 931 beyond 0.5c; the 906× is a paired ratio over those 4489 cells, both arms evaluated on exactly the same cells. What makes it hold up is that it sits on a monotone trend rather than alone, and that 0.005–0.01c (3824 cells) and 0.01–0.02c (8143 cells) independently give 74× and 30×. It is also the band the resolution ladder already refines: at 512^2, where it is 3.4 cells across instead of sub-cell, the interpolator's own near-wall error *falls* 45% while the far field moves 0.6%, so a finer grid would narrow this ratio rather than inflate it.

Neither arm's near-wall skill is observable at $r128$, and that bounds every number in this section. AirfRANS is a boundary-layer mesh, and the mismatch between its node distribution and a uniform raster is not marginal. Over the 200 test cases its clouds carry 35.9 M nodes, 95.4% of them inside

Table 5: Per-band ratio $r_b = \texttt{MSE}_{\text{interp}}(b)/\texttt{MSE}_{\text{Transolver}}(b)$ on the AirfRANS `full` test split, $r128$, 200 cases; $r_b > 1$ means Transolver is better in that band by that factor. Transolver is the mean over the three deployed seeds. Band edges are signed distance in chord units, identical to Table 6. "Cell frac." is the band's share of the $r128$ crop's fluid area; "node frac." its share of the native cloud's nodes; the last column is Transolver's own share of squared u error, whose mirror image is the interpolator's in Table 6. Pooled fluid-cell counts are in the artifact for every band. The accumulator reproduces Table 6 over 95 entries at max relative deviation $6.9{\times}10^{-8}$, and $\sum_b n_b\,\texttt{MSE}_b$ reconstructs the pooled squared error for every arm, channel and band grid (gates **G3/G4**). Source: `results/interpolation/measure_asymmetry.json`, `verdicts.D2`.

band (chord)	cell frac.	node frac.	u	v	p	ν_t	SE share u, Transolver
0–0.005c	0.00138	0.4322	**906.2**	48.3	0.589	393.1	0.0057
0.005–0.01c	0.00118	0.0612	74.1	3.30	2.49	65.6	0.0045
0.01–0.02c	0.00251	0.0619	30.4	1.17	1.58	15.7	0.0094
0.02–0.05c	0.00742	0.0884	5.33	0.391	1.22	4.44	0.0210
0.05–0.15c	0.02942	0.1118	0.572	0.113	0.259	3.70	0.0560
0.15–0.5c	0.15831	0.1311	0.559	0.030	0.031	9.45	0.1847
$> 0.5c$	0.79978	0.1134	**0.312**	0.007	0.004	21.1	**0.7188**

Table 6: Wall-distance decomposition of the interpolator's error on AirfRANS `full` test. R^2_{pc} subtracts each case's own band mean before scoring, so it asks whether spatial structure *within* the band was captured beyond case-level scale. "SE share" is the fraction of the channel's total squared error in that band and is unaffected by the centring choice. The pooled (non-centred) R^2 quoted in the text is higher than R^2_{pc} at every entry; both are in the artifact.

band (chord)	cell frac.	$R^2_{\text{pc}}\,u$	$R^2_{\text{pc}}\,v$	$R^2_{\text{pc}}\,p$	SE share u	SE share v	SE share p
0–0.02c	0.005	0.7539	0.9722	0.9966	**0.924**	**0.898**	0.535
0.02–0.05c	0.007	0.9929	0.9990	0.9986	0.018	0.040	0.238
0.05–0.15c	0.029	0.9991	0.9997	0.9996	0.005	0.030	0.154
0.15–0.5c	0.158	0.9983	0.9999	0.9999	0.017	0.018	0.052
$> 0.5c$	0.800	0.9965	0.9999	1.0000	0.036	0.014	0.022

the $3c{\times}3c$ crop the metric scores. 43.2% of those nodes lie inside $0.005c$ of the wall, a region holding 0.138% of the crop's area—a factor of 312. Cumulatively, 55.5% lie inside $0.02c$ (0.51% of the area, 109.5×) and 64.4% inside $0.05c$ (1.25%, 51.5×); beyond $0.5c$ the asymmetry inverts to 0.14×. The spread across cases is negligible—the node fraction inside $0.05c$ has minimum 0.6045, median 0.6163 and maximum 0.6238—so this is a property of the mesh generator, not of a few cases, and it does not depend on which signed-distance function defines the bands: the grid `sdf` and the cloud's own distance column agree on 99.51% of nodes at the $0.02c$ threshold and 99.90% at $0.05c$ (gate **G6**).

Re-weighting fixes that measure, not the representation both arms are still scored on. To bound the representation, we resample the *rasterised ground truth* at the native nodes and compare its round-trip error against the model's, which is the error the scoring representation commits before any model is involved. Inside $0.005c$ Transolver's per-node u error is 1.16; the $r128$ raster's own is **572.8**. Pooled over the crop the factor is **413**×; inside $0.005c$ it is **495**×. **The scoring representation destroys, in the region holding 43% of the dataset's nodes, structure 495× larger than the model error it is being used to adjudicate.** Every *grid* field number in this paper—ours, and the published rows we set them against—is therefore a statement about far-field skill. This is the half of the problem a re-weighting cannot reach, and it is symmetric: not a concession about the interpolator but a limit on what an $r128$ AirfRANS field metric can say about any method. It is also why Table 1 had to be run rather than the grid refined in place, and why its two memo rows matter: the published $r128$ interpolation output resampled at these nodes scores $3.06{\times}10^6$ on p against the raster's own model-free $3.01{\times}10^6$, i.e. on that channel it is *entirely representation*. (We read this from the in-crop block of the artifact; outside the crop the bilinear sampler clamps query points to the domain boundary, so the outer bands of the full-cloud block are clamp-contaminated and we do not quote them.)

Under refinement the localisation holds and the share statistic dilutes: a pre-registered PARTIAL. Those shares are 128^2 numbers, and the obvious attack is that the $0.02c$ band is thinner than the $0.0234c$ cells it is measured on. We re-ran the whole decomposition at 128^2, 256^2 and 512^2 on the same 800/200 split with the estimator frozen—the kernel weight matrix is a function of the case *names* alone and hashes bit-identically at every rung—and with every band edge fixed in chord units (`scripts/interpolation_resolution_ladder.py`, `results/interpolation/interp_resolution_ladder.json`; the 128^2 rung reproduces Table 2 at relative error 0.00e+00, which gates the run before any finer number is written). **The pre-registered verdict is PARTIAL** and we report it as such. The rule required the near-wall squared-error share to stay at or above 0.85 on *both* u and v at every rung; the shares go 0.924/0.898 at 128^2 to 0.901/0.855 at 256^2 to $\mathbf{0.879}/\mathbf{0.842}$ at 512^2, so v misses the floor at the finest rung. That 0.85 was written before any number existed and was not moved, and no clause of the DISSOLVES branch fires.

The share erodes because its numerator shrank, not because the far field got worse—and that reading is a stronger result than the deployed-resolution table could show. Inside $0.005c$, which is 0.15% of the domain, the absolute u error *falls* 45% under refinement ($466.7 \rightarrow 257.7$) while its R^2 rises $0.507 \rightarrow 0.723$; beyond $0.5c$, which is 80% of the domain, the absolute u error moves 0.6% ($0.0356 \rightarrow 0.0354$). A share is a ratio, so when the dominant term halves and the rest holds, every other band's share rises without any of them degrading. What survives unqualified is everything the objection asserts. At 512^2, where the band is 3.4 cells across rather than sub-cell, **72**% of the u error lies inside $0.005c$: the localisation is *tighter* at fine resolution, not looser. The fraction of the domain reproduced without learning is 0.9949/0.9949/0.9950; every channel still clears $R^2 \geq 0.9996$ beyond $0.05c$ at all three rungs, though we note that the binding entry—p in 0.05–$0.15c$—clears it at 0.99961 at every rung, so that claim rests on a fourth-decimal margin at any resolution; and absolute MSE beyond $0.05c$ is flat within 1.5% across a $16\times$ increase in cell count. Two scope notes travel with the ladder. At 512^2 the raster spacing is only $1.28\times$ the source cloud's far-field nearest-neighbour spacing, so the outer-MSE arm of the rule was pre-declared to be read on the $128 \rightarrow 256$ pair, clear of that limit. And there is no Transolver row at 256^2 or 512^2: the ladder settles whether the *localisation* is an artifact of the deployed grid, and nothing more. Whether the cross-method *advantage* survives at native resolution is not a question a finer raster can answer, which is why Table 1 answers it without one.

The ladder also re-scores the force columns, and they turn out to be ill-conditioned in a way a reader should be told about. By 512^2 the interpolator's C_d relative error rises $2.4\times$ on the median and $2.6\times$ on a 90%-trimmed mean while its *absolute* drag error *falls* ($0.0085 \rightarrow 0.0060$); the untrimmed mean reaches 1.84, but that is one case, because `coefficient_metrics` divides by a rasterised reference drag that collapses and changes sign on that case under refinement ($+0.146$ to -1.7×10^{-4}; the case is named in the artifact). ρ_{C_l} is unmoved at 0.9998 and ρ_{C_d} slips $0.9991 \rightarrow 0.9811$, most of it recoverable by dropping that one case. With no Transolver row at those rungs, and with the rasterised reference integral itself moving under refinement, the 128^2 force ratios cannot be re-evaluated here—only the interpolator's own trend, and only with the metric's ill-conditioning stated.

How we tried to break it. Five checks could have killed the result and did not. *Leakage*: train/test case-name overlap is 0 on all three splits, asserted in the script and verified independently against `manifest.json`. *Near-duplicates*: the minimum nearest-neighbour distance in standardised feature space is 0.189 (median 0.440), and excluding the closest 10% of test cases makes the aggregate *worse*, not better (`mse_p` $75.0 \rightarrow 82.1$), so the headline is not carried by a few near-copies. *Degenerate metric*: predicting each test case from another case's parameters collapses the result to the freestream floor (last row of Table 2; ρ_{C_l} $0.9999 \rightarrow 0.345$, C_l error $1.18\% \rightarrow 365\%$), so the signal is in the parameters, not in the rasterisation or the mask. *Silent drops*: `coefficient_metrics` swallows failed force integrations, so the script asserts $n = 200/200$ (and 196/196 on `aoa`) on every scored variant. *Self-serving solid fill*: filling pressure inside neighbours' solids by nearest-fluid propagation instead of leaving the Delaunay bridge makes surface pressure $14\times$ *worse* ($10989 \rightarrow 153174$), so the one channel the surrogate wins is not flattered by our treatment. Run along the same resolution ladder, that control localises its own sensitivity to the coarse grid: the two defensible fills disagree by $13.7\times$ at 128^2, $5.8\times$ at 256^2 and $1.7\times$ at 512^2, because at $h = 0.0234c$ the bilinear surface stencil straddles the body while at $h = 0.0059c$ it sits in real fluid. We therefore make no claim about surface pressure and grid resolution—the apparent 1.9× degradation under `nearest` reverses to 4.2× the other way under `nearest_all`—and we read our $r128$ surface-pressure number as what it is, a number about the sampler and the solid fill as much as about the flow. Selection never saw test: the cross-validated ranking inside train and the test ranking agree, with kernel ridge and local linear filling the CV top ten and every k-NN configuration 33× behind.

It needs the shape, and it needs few examples. Restricted to (U, α) the baseline collapses (`mse_p` 9491); adding thickness gives 7507; the full seven features give 75.0. The airfoil digits are worth 126× on `mse_p`, so this is not a freestream-lookup table. Conversely, dropping U from the feature metric while keeping the nondimensional rescaling barely moves anything (75.0 → 93.4), so U enters this estimator as pure dimensional scaling. On a train-size ladder with five random subsets per size, 100 **training cases** of parameter interpolation take `mse_p` to 238±65 (worst of five subsets 344) and `mse_v` to 0.0720±0.013, which on the area-uniform measure is past the 7.35M-parameter Transolver trained on all 800 (628.5 and 0.088). At 50 cases the mean already clears `mse_p` (519) but one subset of five does not (735), so the claim we make is 100, not 50. Convergence in `mse_p` has not saturated at 800. The ladder's internal result—an eighth of the training data buys the estimator's full accuracy—is a statement about the interpolator alone and is measure-free. The cross-method comparison inside it is not: it holds on the area-uniform measure only, and is reversed by both Table 4 and Table 1. We report it because the data-efficiency of the estimator is a real property; we do not report it as a win.

The cost asymmetry, and what this does not say. The interpolator has zero parameters but is non-parametric: it carries $800 \times 128^2 \times 4$ float32 = 210 MB of training fields, all resident at inference, against Transolver's 7.35M parameters ($\approx$ 29 MB). That is a real objection to calling it a deployable surrogate. It is not an objection to reporting it as the reference point, which is the only claim we make for it. Nor is this a claim that published AirfRANS results are wrong: it is a claim that they have been read against a reference point nobody ran, through a representation that cannot see the region where the two arms differ. We deliberately do *not* place a row of our own in a cross-method leaderboard, because AirfRANS results are reported in mutually incompatible conventions (standardized-field MSE and official OpenFOAM force labels in the AirfRANS paper, relative-L_2 in the Transolver lineage, physical-unit MSE and self-integrated forces here), so any shared cell would mislead. Table 7 reproduces the canonical AirfRANS-paper baselines as field context and nothing more.

4.2 What the force columns measure, and what the raster costs them

Table 2 carries four force columns, and they are the one part of that table no cell weighting can move. They also do not mean what their names suggest, and the reason is the same representation limit that bounds the volume columns—measured on a different quantity, which makes it a second, independent witness.

What these force correlations measure, and what they do not. Our ρ_{C_l}/ρ_{C_d} correlate the force integrated from the *predicted* field against the force integrated from the *ground-truth* field through the same grid surface-integrator, not agreement with AirfRANS's official OpenFOAM labels. Recomputed against those labels (`scripts/recompute_force_vs_official.py`; 200 cases, 3 seeds; `results/control/force_vs_official.json`), the deployed backbone recovers force *ranking*—$\rho_D = 0.84 \pm 0.01$, $\rho_L = 0.88$—but not magnitude: absolute drag is biased ($\approx$ 11× mean relative error) by the integrator's 1.5-cell surface-sampling offset, and the same integrator on *perfect* fields also yields $\rho_D = 0.84$, so this number is not limited by the prediction. The parameter interpolator makes the same point from the other side: scored through this integrator it ranks the official drag at $\rho = 0.8389$, *indistinguishable from the same integrator applied to the exact ground-truth field* (0.8394), and matching it on lift to four significant figures (0.8758 vs 0.8757). **On this representation, a seven-scalar interpolation of the flow and the flow itself are the same thing as far as official drag ranking is concerned**—which is the force-side statement of the volume-side ceiling in subsection 4.1, on a different quantity and through a different code path.

Where the loss lives, and what is repairable. Rasterising the exact truth to 128^2, scattering it back to the source mesh nodes and integrating with AirfRANS's *own* `force_coefficient` (four pre-registered gates all exactly 0.0) gives $\rho_D = 0.700$ and $\rho_L = 0.9967$—drag *below* our own 0.839, so the raster, not the quadrature, owns the loss. The mechanism is resolution: 68% of drag is viscous and the wall shear lives in a sublayer at $y^+ \approx 5$, so recovering it needs $N \sim 10^5$ cells where lift, set by chord-scale surface pressure, converges at first order over the same ladder: lift is observable from a raster at deployed resolution and drag is not. Repairing what is repairable (`results/control/force_vs_official_multischeme.json`), a wall-extrapolated pressure scheme cuts the ground-truth-field lift magnitude error from 16% to 4.6% (median), and a *control-volume* far-field momentum balance recovers official lift from ground-truth fields almost exactly ($\rho_L = 1.00$, median error $< 1\%$) and from *predicted* fields at $\rho_L = 0.998 \pm 0.001$ with per-seed median magnitude errors of 3.6–3.9%: engineering-grade lift from the surrogate. Drag decomposes diagnostically—on ground-truth fields the control-volume estimator cuts the median drag error from 3.3× to 0.23×, but on predicted fields it reaches only $\approx 2.7\times$ with a degraded ranking, so the residual drag error is the *model's* far-field accuracy, not the integrator. We therefore report lift as quantitatively recovered and make no claim of accurate absolute drag. Two limits travel with this. $\rho_L = 0.998$ is an easy number twice over: α is an input channel and lift here is close to a function of it, and the control-volume estimator recovers lift from circulation through the outer ring, so it depends on the far field and not on the surface at all. And with the solid-adjacent ring masked at $Re \approx 10^6$, the momentum thickness is far below one cell, so the *viscous* component of drag is not merely inaccurate here—it is **unavailable**; every drag statement in this paper is about the pressure-dominated component, which is also why we never report $C_{d,v}$.

4.3 What the physics residual can and cannot do

The obvious response to subsection 4.1 is that the metric should be fixed with physics: score a surrogate by how well its field satisfies the governing equations, and neither the weighting nor the raster has any purchase. We ran that experiment and it does not work. This section reports the measured answer in compressed form; the theorem behind it and the full control set are deferred to a companion paper.

The ground truth fails the check. On 200/200 real AirfRANS test cases the monitored residual of the *exact* field is substantially nonzero ($\|r^\star\|$ mean **0.192**, median 0.133; `results/certificates/residual_floor_realdata.json`), while the physically wrong uniform freestream scores an **exact zero** at every case and every resolution, because a spatially constant field has identically zero finite differences. The objective therefore strictly prefers a wrong field to the truth. This is not an artifact of resolution: rasterising the same truth at 128^2, 256^2 and 512^2 and scoring a region fixed in *physical* units—a reading registered before the run—the floor *rises*, $0.0624 \to 0.0779 \to 0.1067$ while h falls 4×, in **21/24** cases at a fitted exponent $p = -0.387$, with a second independent 16-case five-rung implementation finding it decaying in 0 of 16. Four controls survive, each with its own scope. Method of manufactured solutions on the same monitor gives $p = 2.06$, so the discretisation converges as claimed—but the manufactured field is a potential flow, on which the monitored viscous term vanishes identically, so this control bounds truncation error and is *not* evidence that the operator is consistent with the one that generated the labels. Restoring the omitted closure term $(\partial_j \nu_t)(\partial_j u_i + \partial_i u_j)$ moves each rung's mean floor by -0.01% to $+0.18\%$, and the term is only 3.5–4.8% of the floor. A cubic rather than linear interpolant *raises* the floor by 6–12%, though that control bears on the second-derivative block—5–7% of the floor—and not on the first-derivative block carrying 93–95% of it. And a manufactured analytic solution pushed through the identical rasteriser reproduces the effect $\sim$280× smaller, though its fine end reverses too, so it does not show the pipeline cannot manufacture a rising residual. We name two contributions to the measured size—the mismatch between our Cartesian stencil and the body-fitted discrete balance AirfRANS actually solved, and the reconstruction of that reference onto our grid (at fixed h, thinning the source cloud eightfold raises the floor in 16/16 cases)—and separate neither.

So the residual cannot be descended. Minimising $J = \frac{1}{2}\|R_h(y)\|^2$ directly, with no network in the loop, from the *exact ground truth*—so the standard reply that a surrogate is over-smoothed has nothing to attach to—drives the field error up on **200/200** cases under Armijo-line-searched gradient descent (500 steps, $n = 200$, far-field and near-wall Dirichlet data pinned at truth, so J is monotone non-increasing by construction). J falls to 39.8% of J_0 while the error never returns to its starting value at any step, ending at 1.57× the *entire* error of the deployed Transolver (`mse_u` $0.000 \to 0.499$; $\|R_h\|$ $0.191 \to 0.125$). Started from the deployed Transolver the residual falls 34% while the error rises on **96.5/94.0/98.0**% of cases across three seeds (median $+76.5/+73.9/+86.9\%$; Wilcoxon $p < 10^{-30}$), against a -8 to -25% field-MSE reduction from a *supervised* correction on the same backbone—i.e. the disagreement is with the objective, not with correction as such. The verdict is unchanged under three step rules—Armijo, fixed η swept over 12 decades, and Adam—and is monotone in the *achieved* J: the harder the objective is minimised, the worse the field. Unpinning the boundary data makes the negative stronger (100/99.5/100%), so the pinned arm is the conservative one.

What holds universally is iterate selection, and where the exception lies. We do *not* claim residual descent always harms the field: from a perturbed start it *reduces* error in 18/24 cases before overshooting, and on the far-from-converged dropout-FNO it lowers error on 75% of cases. What holds universally is a statement about **iterate selection**: the error-minimising and residual-minimising iterates differ in 24/24 cases on both arms, and the residual-selected iterate is strictly worse in **24/24**. The exception is measurable in $\rho = \|R_h(\hat{u})\|/\|r^\star\|$—over 200 cases the fraction on which descent helps rises monotonically with ρ, and the deployed backbone sits at median $\rho = 1.13$–1.14 with 88% of cases in the lowest bin, already inside the regime where descent almost never helps. Even there the objective is unusable: the gain is $\approx 1/35$ of the supervised correction's, and there is **no stopping rule**, since J falls monotonically on 100% of cases while the error minimum is interior on 78.5%.

The boundary is the operator, not the problem class. Residual-driven correction is not doomed in general. Lei et al. (2026) take a surrogate prediction as an initial guess for Newton–Krylov correction on *steady* CFD and lower the residual by over seven orders of magnitude *while* reducing field and aerodynamic error; Huang & Perdikaris (2025) report the same on transient rollouts. In those works the residual being driven is the *solver's own* discrete operator, whose root the reference solution is, so its floor at the truth is zero by construction. Ours is the affordable surrogate-side one, and its minimiser is displaced from $u^\star$. The obvious alternative boundaries—steady versus transient, resolved versus under-resolved—are falsified by Lei et al. (2026) being steady and successful. That a PDE residual need not vanish at the ground truth is not ours alone: Zhang et al. (2026) report it for shock physics and conclude a physics-informed loss "must be defined relative to the residual present in the ground-truth data"—a remedy requiring $R_h(u^\star)$, which a deployment-time monitor does not have. What the residual *can* do is rank: the case-level residual–error Spearman correlation is positive on every backbone, seed and split we have measured. That is a detector, not a metric, and it is reported in the companion paper rather than here, because none of it repairs the measurement problem this paper is about.

5 Limitations

- **Every *grid* field number in this paper is scored on the $r128$ raster, and that representation cannot adjudicate the boundary layer for either arm.** The rasterised ground truth, resampled at the native cloud nodes, commits a round-trip error 413× larger than Transolver's own per-node error pooled over the scored crop, and 495× larger inside $0.005c$, where 43% of the dataset's nodes live. Two things follow. First, the *weighting* of that raster is not a neutral choice, and we therefore report both weightings rather than one (Table 4); the ranking reverses between them. Second, no re-weighting reaches the representation, which is why Table 1 removes the raster instead of refining it. A frozen $128^2/256^2/512^2$ ladder settles that the *localisation* is not an artifact of the deployed grid—the domain fraction reproduced without learning, the R^2 floor beyond $0.05c$ and the absolute outer-region MSE are flat across a 16× increase in cell count, and at 512^2 the error is *more* tightly localised—while the near-wall *share* eases from 0.92/0.90 to 0.88/0.84 and misses a pre-declared 0.85 floor on v at the finest rung, so the registered verdict is PARTIAL and we report it as PARTIAL. Relatedly, Table 3 divides by variances computed on the $r128$ raster rather than in point space, so it is faithful in spirit but not byte-identical to the training normaliser; the ratios between rows are unaffected, since the divisor is shared.
- **What the native-resolution head-to-head does not settle.** Four scopes, each of which a reviewer is entitled to press. *(i)* The interpolator's hyperparameters are the **grid**-cross-validated

ones; re-selecting them under the node measure would require 800×800 cloud-to-cloud transfers and was not run. The least-squares oracle bounds what that can buy near the wall—a factor of a few against a measured 21× for the whole family, and 344× for the published estimator—but **P1**'s scope is correctly "this estimator", not "every parameter-space estimator". *(ii)* The least-squares family bound is $n = 3$ cases, one channel and one band, for the reason given in subsection 4.1: it is a bound only where the node count far exceeds the 800 free parameters of the projection. *(iii)* No body-fitted or wall-aligned interpolator is tested, deliberately (subsection 4.1); that is a different method and our result does not speak to it. *(iv)* The `full` split only; the head-to-head was not re-run at native nodes on `reynolds` or `aoa`.

- **The far-field u entry disagrees between the two measures and we cannot yet say why.** On the grid the interpolator is 3.2× better than the surrogate on u beyond $0.5c$; at the native nodes it is 3.8× worse, and the pre-registered localisation verdict is CONFIRMED in the first case and PARTIAL in the second. Resampling the published grid output at those nodes accounts for only 1.27× of the swing, so "it was the raster" is not supported either. The sub-band decomposition that would apportion the remainder between where the band is sampled and what the reference is has not been run. We report the disagreement rather than the convenient half of it.

- **The interpolator is a reference point, not a method.** It has zero parameters but carries 210 MB of training fields resident at inference, and it is deterministic given the split, so its rows have no seed variance to report (the train-subset ladder uses five random subsets per size for $n < 800$). We make no deployability claim for it.

- **Grid resolution, near-wall blindness, and what that costs drag.** A uniform 128^2 Cartesian grid cannot resolve a $Re \approx 10^6$ boundary layer, so wall quantities are approximate and the absolute grid MSE values are not comparable to body-fitted solvers. With the solid-adjacent ring masked, the momentum thickness is far below one cell, so the *viscous* component of drag is unavailable on this representation and every drag statement here concerns the pressure-dominated component.

- **Eddy-viscosity accuracy is backbone-dependent.** Against ν_t's own fluid-masked variance the channel is well predicted on a grid FNO backbone ($R^2 = 0.996$) but only moderately on a message-passing GNN ($R^2 = 0.67$–0.70). Because $\nu_t \approx 5.2\,\nu$ supplies $\approx 84\%$ of ν_{eff}, a backbone with a weak ν_t carries that error straight into the monitored residual of subsection 4.3—that operator is only as turbulent as the closure the surrogate predicts.

- **The resolution ladder has its own ceiling, and we do not explain the rise.** The floor's growth is measured over 128^2–512^2, and at the finest level the raster spacing is only 1.27× the source cloud's far-field nearest-neighbour spacing, so a still finer level would begin differentiating the interpolant rather than sampling the solution. The $128^2 \to 256^2$ leg is clear of that limit and already shows the rise, but we claim no asymptotic rate—only that the floor does not decay over the range a 128^2-trained surrogate is deployed in. We also stop short of naming the mechanism: holding h fixed and thinning the source cloud eightfold raises the floor in 16/16 cases ($\mathrm{d}\log\|r^\star\|/\mathrm{d}\log s = +0.55 \pm 0.27$), which rules out attributing the growth to operator provenance alone, and the registered representation-error model (predicted exponent 1.64, rise 4.8–8.0×) is refuted by the measured 1.8×.

- **Scope of the descent result.** The n=24 arms use Adam with per-channel scaling, so the field error rises overall but not monotonically and those claims are about endpoints rather than paths; the n=200 Armijo arm closes this. The headline arm also freezes ν_t: descending the eddy viscosity alongside (u, v, p) quarters the velocity damage, but only by dumping the residual into the closure—on the deployed backbone (seed 0) `mse_nut` rises seven orders of magnitude ($6.3 \times 10^{-9} \to 0.115$) while the field error still worsens on 82.5% of cases—so we freeze it in the headline and report the four-channel arm as a control. The full evidence, including the resolution ladder, the four artifact controls and the theorem, is deferred to a companion paper.

- **One benchmark, two dimensions.** Everything here is AirfRANS and everything here is 2-D. The mechanism is not airfoil-specific—any benchmark that distributes wall-clustered clouds and scores predictions on a resampled uniform grid inherits both the weighting ambiguity and the representation ceiling, and the released harness computes both diagnostics for an arbitrary cloud-plus-raster pair—but we have measured it on one dataset. Extending to 3-D external aerodynamics (Ashton et al., 2024; Elrefaie et al., 2024) requires multi-TB datasets beyond a single-GPU budget.

6 Conclusion

What a neural flow surrogate buys on AirfRANS is near-wall representation, and no member of the interpolation family reaches it. Scored per node on the dataset's own cloud, with no rasterisation anywhere in the comparison path, a Gaussian kernel-ridge interpolation of seven scalars read off the case name loses to a matched-budget Transolver on every channel—144× on u, 134× on v, 4.9× on p, 18× on ν_t, and 200/200 cases on three of the four. An oracle handed the test answer and allowed the best single training field is 344× worse inside the first 0.005 chord; the least-squares optimum over the span of all 800 training fields, the exact lower bound for any weighting of that family, is still 21× worse there. The gap at the wall is representational, and the mechanism is measured: inside $0.005c$, where 41% of every AirfRANS cloud lives, a query lands on average 7.8 times further from the training case's wall than from its own, with 35% of the weight mass inside the training airfoil.

The published protocol says the opposite, and what decides it is a convention nobody states. On the area-uniform 128^2 raster the field tables are reported on, the same interpolator gives 8.4× *lower* volume-pressure error than the same surrogate. Re-weight the identical predictions on the identical cells by native node count and that becomes 0.44×; per node it is 0.21×. AirfRANS places 64.4% of its mesh inside $0.05c$ of the wall against 1.25% of the raster's area, and no published comparison states which weighting it uses. Nor can the raster settle the matter by being refined in place: resampled at the native nodes its own round-trip error exceeds the surrogate's by 413× pooled and 495× near the wall, so *an* $r128$ *field metric on this benchmark is a measurement of far-field skill for every method scored on it.* What both measures do agree on is where each method's error lives: the band ratio runs 906× → 0.31× on the grid and 4052× → 3.8× at the nodes, with 71.9% of the surrogate's own squared error beyond $0.5c$ against the interpolator's 3.6%, which reproduces 98.7% of the domain at $R^2 \geq 0.9996$ from seven numbers in a file name. A surrogate earns the boundary layer—precisely where the benchmark's own authors located their models' failure without quantifying it.

Fixing the metric with physics does not work, and the reason is measurable. On 200/200 AirfRANS cases the monitored residual of the exact field is substantially nonzero (mean 0.192) while the physically wrong uniform freestream scores an exact zero, and refining the grid makes the floor *larger*. Descending that residual from the exact ground truth, with no network in the loop, drives the field error up on **200/200** cases. The boundary is the operator, not the problem class: Newton–Krylov corrections succeed on steady CFD precisely because the residual they drive is the solver's own, whose floor at the truth is zero by construction.

The durable contribution is a measurement of what a standard scoring path costs, and three things follow from it that any author in this line can act on. Report a non-learned reference point alongside the model, because parameter interpolation takes fifteen minutes of CPU time and localises whatever advantage remains. State the cell weighting, because on this benchmark that choice moves a pressure ratio by a factor of 19 and reverses its sign. And before reading a field metric as evidence about near-wall skill, measure the scoring representation's own error at the nodes—if it exceeds the model's, the comparison is being decided by the instrument. Anyone building a surrogate-side physics check should apply the same discipline to their operator: measure its floor on their own ground truth first, because if it is not near zero the objective is not usable.

A Published AirfRANS baselines

Table 7 reproduces the canonical AirfRANS-paper baselines as field context only. Every figure and table supporting a claim the main text makes stays beside that claim.

Table 7: Published AirfRANS `full`-task baselines from Bonnet et al. (2022), as field context only. Volume MSE on standardized fields ($\times 10^{-2}$); surface $\bar{p}$ ($\times 10^{-1}$); force relative errors (%); ρ = Spearman rank correlation vs. *official* force labels. Our rows are *not* comparable (physical-unit MSE, self-integrated forces; subsection 4.2). The near-zero/negative ρ_{C_d} is the benchmark's known drag-ranking difficulty, which subsection 4.2 traces to the representation rather than to the quadrature.

method	$\bar{u}_x$	$\bar{p}$	surf. $\bar{p}$	C_L err	C_D err	ρ_{C_l} / ρ_{C_d}
MLP	0.95	0.74	1.13	0.77	4.29	0.913 / −0.117
GraphSAGE	0.83	0.66	0.66	0.52	4.05	0.965 / −0.303
PointNet	3.50	1.15	0.93	0.74	14.64	0.938 / −0.022
Graph U-Net	1.52	0.66	0.39	0.49	10.39	0.967 / −0.138

Code and data availability

The complete source, training/evaluation harnesses, and per-claim reproduction map (`docs/REPRODUCE.md`) are released as the open-source package `neuroforge-cfd` (`https://github.com/ali-kin4/neuroforge-cfd`; archived at Zenodo, DOI 10.5281/zenodo.21277928). Every headline number maps to a committed script and result file, and a top-level manifest (`results/MANIFEST.json`) records seeds, environment, and SHA-256 hashes for hash-drift detection. The benchmark is public (Bonnet et al., 2022). The package is CPU-first and runs end-to-end with zero downloads via a synthetic data generator; the GPU runs (backbone training) are documented with exact commands and expected outputs. The two diagnostics this paper turns on—the node-versus-area weighting of a scoring raster, and that raster's own round-trip error at the source nodes—are implemented as reusable routines that take an arbitrary point cloud and raster specification, so the check is portable to any benchmark of this form.

Use of AI-assisted development tools

Substantial portions of the research software (model implementations, training/evaluation harnesses, and analysis scripts) were developed with the assistance of a generative-AI coding tool (Claude, Anthropic; Claude Opus 4.8 and Claude Fable 5 models, used through the Claude Code environment), operating under the authors' direction. The authors specified, reviewed, and validated all code; every reported number is produced by a committed script in the public repository and verified against an artifact manifest of SHA-256 hashes (`results/MANIFEST.json`), and the full test suite accompanies the release. The scientific questions, experimental design, pre-registered hypotheses, and interpretation of results are the authors' own.

CRediT author contribution statement

Ali Jabbary: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Kasra Ghanavati:** Validation, Writing – review & editing.

Declaration of competing interests

The authors declare no competing financial or personal interests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist in drafting and editing the manuscript text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The use of AI assistance in developing the research software is described above.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors. All experiments reported here were run on a single consumer workstation (one NVIDIA RTX 4070 Ti for backbone training; every audit-side measurement is CPU-only), which is part of why the reproduction package is CPU-first.

Acknowledgements

We thank the authors of AirfRANS (Bonnet et al., 2022) for releasing their dataset, benchmark splits, and reference force labels publicly, and—decisively for this work—the native, unrasterised point clouds without which the head-to-head of Table 1 could not have been run. We also thank the maintainers of the open-source scientific Python and PyTorch ecosystems on which the released package depends. Assistance from generative AI tools in preparing the manuscript text is declared separately above.

References

Constantin Ahlmann-Eltze, Wolfgang Huber, and Simon Anders. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods*, 22(8):1657–1661, 2025. doi: 10.1038/s41592-025-02772-6.

Neil Ashton, Danielle C. Maddix, Samuel Gundry, and Parisa M. Shabestari. AhmedML: High-fidelity computational fluid dynamics dataset for incompressible, low-speed bluff body aerodynamics. *arXiv preprint arXiv:2407.20801*, 2024. doi: 10.48550/arXiv.2407.20801.

Roland Becker and Rolf Rannacher. An optimal control approach to a posteriori error estimation in finite element methods. *Acta Numerica*, 10:1–102, 2001. doi: 10.1017/S0962492901000010.

Florent Bonnet, Jocelyn Ahmed Mazari, Paola Cinnella, and Patrick Gallinari. AirfRANS: High fidelity computational fluid dynamics dataset for approximating reynolds-averaged navier-stokes solutions. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022. arXiv:2212.07564.

Mohamed Elrefaie, Florin Morar, Angela Dai, and Faez Ahmed. DrivAerNet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. arXiv:2406.09624.

Federico Errica, Marco Podda, Davide Bacciu, and Alessio Micheli. A fair comparison of graph neural networks for graph classification. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1912.09893.

Kawin Ethayarajh, Yejin Choi, and Swabha Swayamdipta. Understanding dataset difficulty with $\mathcal{V}$-usable information. In *International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pp. 5988–6008, 2022. Outstanding Paper; arXiv:2110.08420.

Shi Feng, Eric Wallace, and Jordan Boyd-Graber. Misleading failures of partial-input baselines. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 5533–5538, 2019. doi: 10.18653/v1/P19-1554. arXiv:1905.05778.

Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*, 2019. Best Long Paper; arXiv:1907.06902.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 107–112, 2018. doi: 10.18653/v1/N18-2017. arXiv:1803.02324.

Birgit Hillebrecht and Benjamin Unger. Rigorous a posteriori error bounds for PDE-defined PINNs. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1):1583–1593, 2025. doi: 10.1109/TNNLS.2023.3335837.

Jun-Ting Hsieh, Shengjia Zhao, Stephan Eismann, Lucia Mirabella, and Stefano Ermon. Learning neural PDE solvers with convergence guarantees. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1906.01200.

Xinquan Huang and Paris Perdikaris. PhysicsCorrect: A training-free approach for stable neural PDE simulations. *arXiv preprint arXiv:2507.02227*, 2025. doi: 10.48550/arXiv.2507.02227. AAAI 2026.

Prashant K. Jha. Residual-based error corrector operator to enhance accuracy and reliability of neural operator surrogates of nonlinear variational boundary-value problems. *Computer Methods in Applied Mechanics and Engineering*, 419:116595, 2024. doi: 10.1016/j.cma.2023.116595. arXiv:2306.12047.

Mingcheng Lei, Weishao Tang, Yufei Zhang, and Haixin Chen. Reliable and efficient steady CFD from surrogate predictions through Newton–Krylov correction. *arXiv preprint arXiv:2608.04400*, 2026. doi: 10.48550/arXiv.2608.04400. Submitted 5 August 2026.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.

Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for pdes on general geometries. *Journal of Machine Learning Research*, 24(388): 1–26, 2023. arXiv:2207.05209.

Phillip Lippe, Bastiaan S. Veeling, Paris Perdikaris, Richard E. Turner, and Johannes Brandstetter. PDE-refiner: Achieving accurate long rollouts with neural PDE solvers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2308.05732.

Huakun Luo, Haixu Wu, Hang Zhou, Lanxiang Xing, Yichen Di, Jianmin Wang, and Mingsheng Long. Transolver++: An accurate neural solver for PDEs on million-scale geometries. *arXiv preprint arXiv:2502.02414*, 2025. doi: 10.48550/arXiv.2502.02414.

Tanya Marwah, Ashwini Pokle, J. Zico Kolter, Zachary C. Lipton, Jianfeng Lu, and Andrej Risteski. Deep equilibrium based neural operators for steady-state PDEs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2312.00234.

Nick McGreivy and Ammar Hakim. Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. *Nature Machine Intelligence*, 6:1256–1269, 2024. doi: 10.1038/s42256-024-00897-5.

Amartya Mukherjee, Maxwell Fitzsimmons, David C. Del Rey Fernández, and Jun Liu. Rigorous error certification for neural PDE solvers: From empirical residuals to solution guarantees. *arXiv preprint arXiv:2603.19165*, 2026. doi: 10.48550/arXiv.2603.19165.

Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. Hypothesis only baselines in natural language inference. In *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics (*SEM)*, pp. 180–191, 2018. doi: 10.18653/v1/S18-2023. arXiv:1805.01042.

Nestor V. Queipo, Raphael T. Haftka, Wei Shyy, Tushar Goel, Rajkumar Vaidyanathan, and P. Kevin Tucker. Surrogate-based analysis and optimization. *Progress in Aerospace Sciences*, 41(1):1–28, 2005. doi: 10.1016/j.paerosci.2005.02.001.

Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

Rishikesh Ranade, Mohammad Amin Nabian, Kaustubh Tangsali, Alexey Kamenev, Oliver Hennigh, Ram Cherukuri, and Sanjay Choudhry. DoMINO: A decomposable multi-scale iterative neural operator for modeling large scale engineering simulations. *arXiv preprint arXiv:2501.13350*, 2025. doi: 10.48550/arXiv.2501.13350. NVIDIA.

Stephan Rasp, Stephan Hoyer, Alexander Merose, Ian Langmore, Peter Battaglia, Tyler Russel, Alvaro Sanchez-Gonzalez, Vivian Yang, Rob Carver, Shreya Agrawal, Matthew Chantry, Zied Ben Bouallegue, Peter Dueben, Carla Bromberg, Jared Sisk, Luke Barrington, Aaron Bell, and Fei Sha. WeatherBench 2: A benchmark for the next generation of data-driven global weather models. *Journal of Advances in Modeling Earth Systems*, 16(6):e2023MS004019, 2024. doi: 10.1029/2023MS004019.

Mattia Rigotti, Nicholas Thumiger, and Thomas Frick. GIST: Gauge-invariant spectral transformers for scalable graph neural operators. *arXiv preprint arXiv:2603.16849*, 2026. doi: 10.48550/arXiv.2603.16849.

Jan Scherz, Derrick Hines, and Philipp Bekemeyer. Evaluation of state-of-the-art deep learning architectures for aerodynamical predictions. *arXiv preprint arXiv:2607.13866*, 2026. doi: 10.48550/arXiv.2607.13866.

Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 53681–53705, 2024. arXiv:2402.02366.

S. Zhang, M. Mallon, M. Luo, J. Thiyagalingam, P. Tzeferacos, R. Bingham, and G. Gregori. Data-driven modeling of shock physics by physics-informed MeshGraphNets. *arXiv preprint arXiv:2602.14918*, 2026. doi: 10.48550/arXiv.2602.14918. Submitted 16 February 2026.